

Database Systems (CS2005)

Week – 14

Instructor: **Basit Ali**

CHAPTER 20

Introduction to Transaction Processing Concepts and Theory

Introduction

- Transaction
 - A transaction is a unit of program execution that accesses and possibly updates various data items.
- Transaction processing systems
 - Systems with large databases and hundreds of concurrent users
 - Require high availability and fast response time

Transaction Processing

- Single-user DBMS
 - At most one user at a time can use the system
 - Example: home computer
- Multiuser DBMS
 - Many users can access the system (database) concurrently
 - Example: airline reservations system
- Multiprogramming
 - Allows operating system to execute multiple processes concurrently
 - Executes commands from one process, then suspends that process and executes commands from another process, etc.

Transaction and System Concepts

- System must keep track of when each transaction starts, terminates, commits, and/or aborts
 - BEGIN_TRANSACTION
 - READ or WRITE
 - END_TRANSACTION
 - COMMIT_TRANSACTION
 - ROLLBACK (or ABORT)

Transaction and System Concepts (cont'd.)

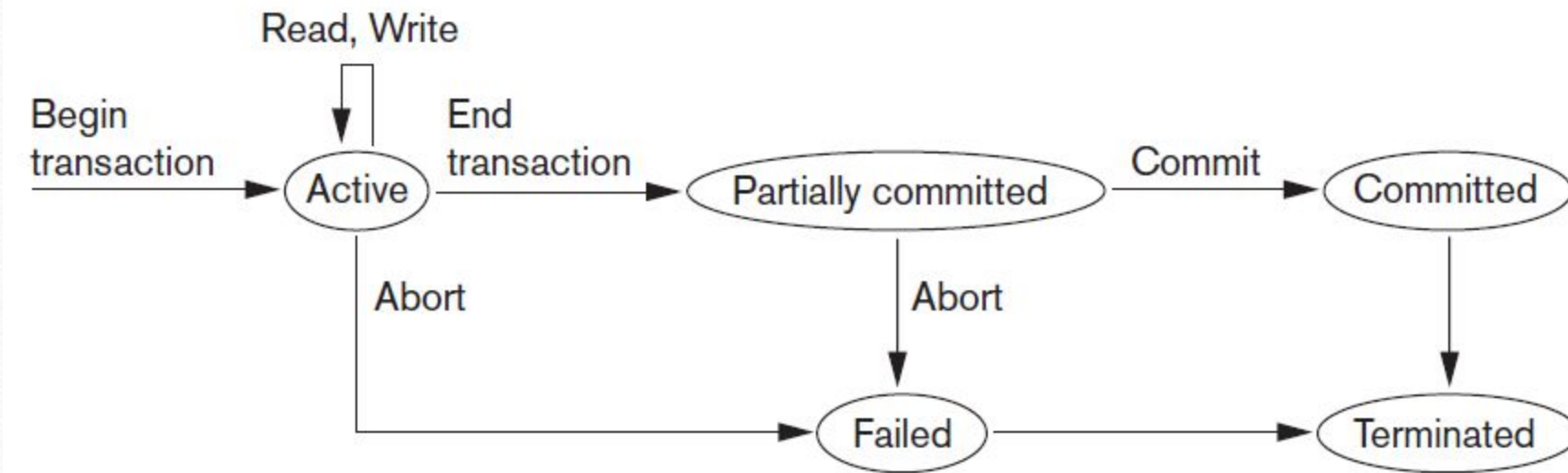


Figure 20.4 State transition diagram illustrating the states for transaction execution

Commit Point of a Transaction

- Occurs when all operations that access the database have completed successfully
 - And effect of operations recorded in the log
- Transaction writes a commit record into the log
 - If system failure occurs, can search for transactions with recorded start_transaction but no commit record
- Force-writing the log buffer to disk
 - Writing log buffer to disk before transaction reaches commit point

Transaction Processing

- **Interleaved Processing:** Interleaved processing refers to switching between tasks at regular intervals, creating the appearance of simultaneous execution. This method allocates a time slice to each task and switches back and forth, managing multiple tasks by sharing resources and processing time.
- **Example:** In a single-core processor running two tasks (Task A and Task B), the processor might execute Task A for a short time, then switch to Task B, then back to Task A, and so on. This gives the impression that both tasks are running concurrently, although only one task is actually processed at any given moment.

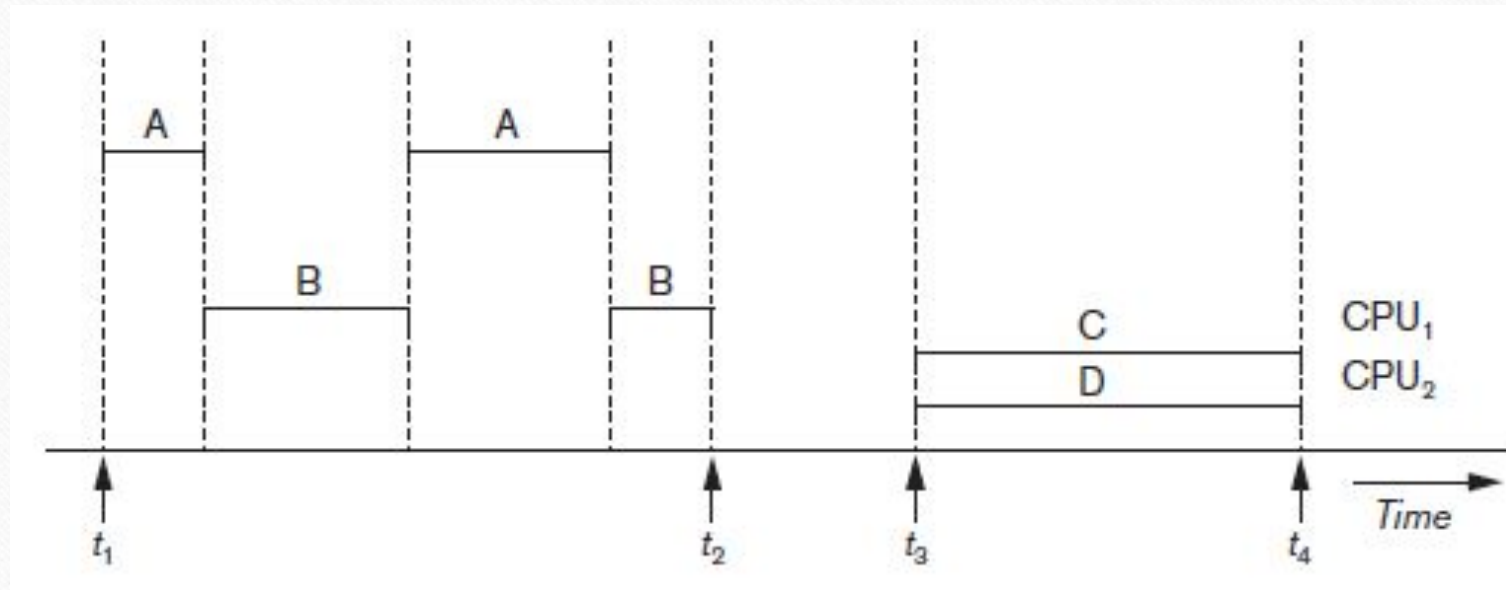
Transaction Processing

- **Parallel Processing:** Parallel processing involves executing multiple tasks or processes simultaneously. This requires multiple processing cores or processors, where each core handles a separate task, allowing real concurrency.
- **Example:** In a multi-core processor, each core could handle a different task at the same time. For example, Core 1 might process Task A, and Core 2 might simultaneously process Task B, resulting in true parallelism and faster execution.

Interleaved V/s Parallel Processing

Feature	Interleaved Processing	Parallel Processing
Definition	Time-slicing approach where tasks are switched frequently, giving an appearance of simultaneous execution.	Multiple tasks are executed at the same time on different cores or processors.
Concurrency Type	Pseudo-concurrency (appears concurrent, but tasks are not truly simultaneous)	True concurrency (tasks run simultaneously)
Requirement	Can work on a single-core processor	Requires multi-core or multiple processors
Example	Task A and Task B are executed alternately in short intervals on a single-core processor.	Task A runs on Core 1, while Task B runs on Core 2 simultaneously.
Performance	Slower, as tasks share the same resources and time slices	Faster, as tasks have dedicated resources and are processed in parallel
Best Suited For	Systems with limited processing resources or multitasking on a single-core CPU	High-performance computing where tasks can be truly parallelized

Interleaved V/s Parallel Processing



Database Items

- Database represented as collection of named data items
- Size of a data item called its granularity
- Data item
 - Record
 - Disk block
 - Attribute value of a record

Read and Write Operations

- `read_item(X)`
 - Reads a database item named X into a program variable named X
 - Process includes finding the address of the disk block, and copying to and from a memory buffer
- `write_item(X)`
 - Writes the value of program variable X into the database item named X
 - Process includes finding the address of the disk block, copying to and from a memory buffer, and storing the updated disk block back to disk

Read and Write Operations (cont'd.)

- Read set of a transaction
 - Set of all items read
- Write set of a transaction
 - Set of all items written

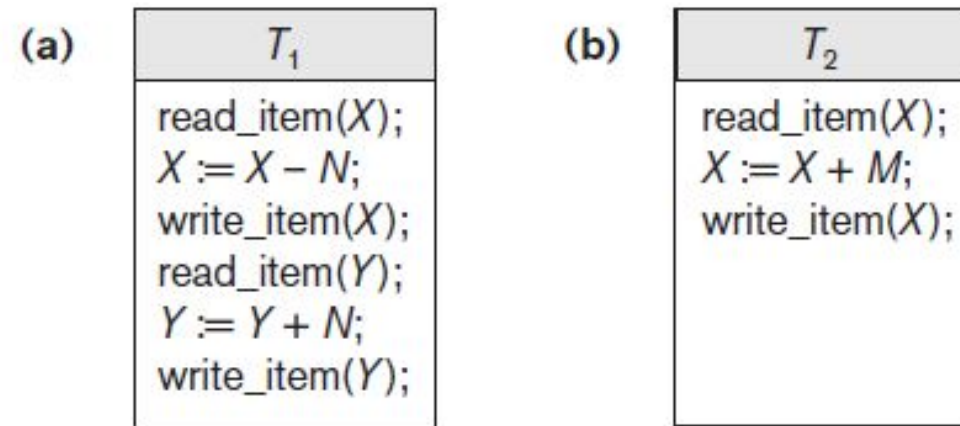


Figure 20.2 Two sample transactions (a) Transaction T_1 (b) Transaction T_2

DBMS Buffers

- DBMS will maintain several main memory data buffers in the database cache
- When buffers are occupied, a buffer replacement policy is used to choose which buffer will be replaced
 - Example policy: least recently used

Concurrency Control

- Transactions submitted by various users may execute concurrently
 - Access and update the same database items
 - Some form of concurrency control is needed
- The lost update problem
 - Occurs when two transactions that access the same database items have operations interleaved
 - Results in incorrect value of some database items

The Lost Update Problem

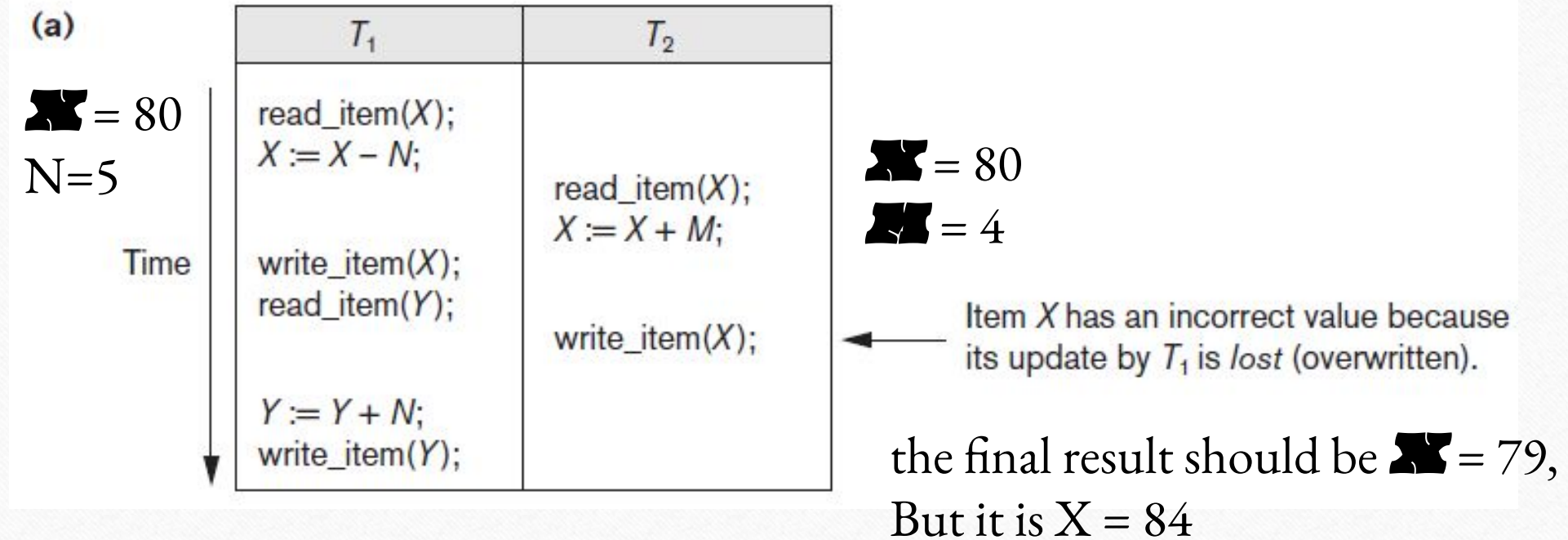


Figure 20.3 Some problems that occur when concurrent execution is uncontrolled (a) The lost update problem

The Lost Update Problem

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	read(bal _x)	100
t ₃	read(bal _x)	bal _x = bal _x + 100	100
t ₄	bal _x = bal _x - 10	write(bal _x)	200
t ₅	write(bal _x)	commit	90
t ₆	commit		90

Figure 20.3 Some problems that occur when concurrent execution is uncontrolled (a) The lost update problem

Temporary Update Problem (also known as the Dirty Read Problem or Uncommitted Dependency Problem)

- The Dirty Read Problem occurs when a transaction reads data that has been modified by another uncommitted transaction. If the modifying transaction rolls back, the data read becomes invalid or "dirty."
- Example: Transaction T1 updates a record but has not committed. Transaction T2 reads this uncommitted data. If T1 rolls back, T2 has read an invalid (dirty) value.
- Scenario:
 - Transaction T1:** Updates balance from \$500 to \$700 (uncommitted)
 - Transaction T2:** Reads balance as \$700
 - T1 Rollback:** Balance reverts to \$500, leaving T2 with a "dirty" read.

The Temporary Update Problem OR Dirty Read Problem OR Uncommitted Dependency Problem

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal _x)	100
t ₃		bal_x = bal_x + 100	100
t ₄	begin_transaction	write(bal _x)	200
t ₅	read(bal _x)	⋮	200
t ₆	bal_x = bal_x - 10	rollback	100
t ₇	write(bal _x)		190
t ₈	commit		190

Figure 20.3 (cont'd.) Some problems that occur when concurrent execution is uncontrolled (b) The temporary update problem

The Incorrect Summary Problem

(c)

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

Figure 20.3 (cont'd.) Some problems that occur when concurrent execution is uncontrolled (c) The incorrect summary problem

The Incorrect Summary Problem OR Inconsistent Analysis Problem

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	100	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	25	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

Figure 20.3 (cont'd.) Some problems that occur when concurrent execution is uncontrolled (c) The incorrect summary problem

The Unrepeatable Read Problem (Fuzzy Read)

- The Unrepeatable Read problem, also known as Fuzzy Read, is a concurrency issue in databases where a transaction reads the same row twice and gets different values each time because another transaction modifies that row between the reads
- This can lead to inconsistent data views within the transaction, causing unexpected results.

Fuzzy Read (Example)

Step	Transaction	Action	Account Balance Observed	Explanation
1	T1	<code>SELECT balance FROM accounts WHERE account_id = 101</code>	\$500	T1 reads the initial balance of \$500.
2	T2	<code>UPDATE accounts SET balance = balance - 100 WHERE account_id = 101</code>	\$400	T2 withdraws \$100 and updates the balance to \$400.
3	T2	<code>COMMIT</code>	\$400	T2 commits the update, making the new balance of \$400 visible to other transactions.
4	T1	<code>SELECT balance FROM accounts WHERE account_id = 101</code>	\$400	T1 reads the balance again and now sees \$400, not the original \$500 it saw in Step 1.

The Phantom Read Problem

- Transaction T executes a query that retrieves a set of tuples from a relation satisfying a certain predicate
- T re-executes the query at a later time, but finds that the retrieved set contains an additional (phantom) that has been inserted by another transaction in the meantime.
- T receives different tuples for the same query

The Phantom Read Problem (Example)

Time	Transaction	Action	Resulting Count of Employees	Explanation
T1	T1	BEGIN TRANSACTION	-	T1 starts a transaction.
T2	T1	SELECT COUNT(*) FROM employees WHERE department_id = 101 AND salary > 50000	5	T1 counts 5 employees in department 101 with a salary > \$50,000.
T3	T2	BEGIN TRANSACTION	-	T2 starts a transaction.
T4	T2	INSERT INTO employees (name, department_id, salary) VALUES ('John', 101, 60000)	6	T2 inserts a new employee (John) with salary \$60,000 in department 101.
T5	T2	COMMIT	-	T2 commits the insertion, making John visible to other transactions.
T6	T1	SELECT COUNT(*) FROM employees WHERE department_id = 101 AND salary > 50000	6	T1 re-counts and sees 6 employees now, instead of the original 5.
T7	T1	COMMIT	-	T1 commits, ending its transaction.

Why Recovery is Needed

- Committed transaction
 - Effect recorded permanently in the database
- Aborted transaction
 - Does not affect the database
- Types of transaction failures
 - Computer failure (system crash)
 - Transaction or system error
 - Local errors or exception conditions detected by the transaction

Why Recovery is Needed (cont'd.)

- Types of transaction failures (cont'd.)
 - Concurrency control enforcement
 - Disk failure
 - Physical problems or catastrophes
- System must keep sufficient information to recover quickly from the failure
 - Disk failure or other catastrophes have long recovery times

The System Log

- System log keeps track of transaction operations
- Sequential, append-only file
- Not affected by failure (except disk or catastrophic failure)
- Log buffer
 - Main memory buffer
 - When full, appended to end of log file on disk
- Log file is backed up periodically
- Undo and redo operations based on log possible

DBMS-Specific Buffer Replacement Policies

- Page replacement policy
 - Selects particular buffers to be replaced when all are full
- Domain separation (DS) method
 - Each domain handles one type of disk pages
 - Index pages
 - Data file pages
 - Log file pages
 - Number of available buffers for each domain is predetermined

DBMS-Specific Buffer Replacement Policies (cont'd.)

- Hot set method
 - Useful in queries that scan a set of pages repeatedly
 - Does not replace the set in the buffers until processing is completed
- The DBMIN method
 - Predetermines the pattern of page references for each algorithm for a particular type of database operation
 - some queries may reference the same file twice, so there would be a locality set for each file instance needed in the query
 - Calculates locality set using Query Locality Set Model (QLSM)

Desirable Properties of Transactions

- ACID properties
 - Atomicity
 - Transaction performed in its entirety or not at all
 - Consistency preservation
 - Takes database from one consistent state to another
 - Isolation
 - Not interfered with by other transactions
 - Durability or permanency
 - Changes must persist in the database

Desirable Properties of Transactions (cont'd.)

- Levels of isolation
 - Level 0 isolation does not overwrite the dirty reads of higher-level transactions
 - Level 1 isolation has no lost updates
 - Level 2 isolation has no lost updates and no dirty reads
 - Level 3 (true) isolation has repeatable reads
 - In addition to level 2 properties
 - Snapshot isolation
 - transaction sees the data items that it reads based on the committed values of the items in the database snapshot (or database state) when the transaction starts. Snapshot isolation will ensure that the phantom record problem does not occur

Characterizing Schedules Based on Recoverability

- Schedule or history
 - Order of execution of operations from all transactions
 - Operations from different transactions can be interleaved in the schedule
- Total ordering of operations in a schedule
 - For any two operations in the schedule, one must occur before the other

Some Definitions

- Schedule
 - A sequence of the operations by a set of concurrent transactions that preserves the order of the operations in each of the individual transactions.
- Serial Schedule
 - A schedule where the operations of each transaction are executed consecutively without any interleaved operations from other transactions.
- Non-serial Schedule
 - A schedule where the operations from a set of concurrent transactions are interleaved.
- Serializability
 - find non-serial schedules that allow transactions to execute concurrently without interfering with one another, and thereby produce a database state that could be produced by a serial execution.

Example

Schedule Type	Description	Example
Schedule	A sequence of operations for concurrent transactions, where each transaction's order of operations is preserved.	T1: Read(X), Write(X + 10) T2: Read(Y), Write(Y + 20)
Serial Schedule	Each transaction runs completely before the next transaction starts, maintaining isolation.	Serial Schedule Example: T1: Read(X), Write(X + 10) T2: Read(Y), Write(Y + 20)
Non-serial Schedule	Operations of transactions are interleaved, meaning T1 and T2 operations are mixed.	Non-serial Schedule Example: T1: Read(X) T2: Read(Y) T1: Write(X + 10) T2: Write(Y + 20)
Serializability	A non-serial schedule that produces the same result as a serial schedule, making it safe for concurrent execution.	Serializable Non-serial Schedule: T1: Read(X) T2: Read(Y) T1: Write(X + 10) T2: Write(Y + 20) (serializable)

i.e. (in essence)

- If a set of transactions executes concurrently,
- We say that the (non-serial) schedule is correct
 - If it produces the same result as some serial execution.
- Such a schedule is called *serializable*.
- To prevent inconsistency from transactions interfering with one another... it is essential to guarantee serializability of concurrent transactions.

Characterizing Schedules Based on Recoverability (cont'd.)

- Two conflicting operations in a schedule
 - Operations belong to different transactions
 - Operations access the same item X
 - At least one of the operations is a write_item(X)
- Two operations conflict if changing their order results in a different outcome
- Read-write conflict
- Write-write conflict

Read-write conflict & Write-write conflict

Conflict Type	Description	Example
Read-Write Conflict	Happens when a transaction reads a data item that another transaction is in the process of updating.	T1: Read(X) T2: Write(X) T1: Write(X + 10) Here, T1 reads X before T2 updates it, potentially causing T1 to use stale data.
Write-Write Conflict	Occurs when two transactions attempt to write to the same data item without coordination.	T1: Write(X + 10) T2: Write(X + 20) Here, T1 and T2 both write to X independently, leading to uncertainty about which final value X holds.

Characterizing Schedules Based on Recoverability (cont'd.)

- Recoverable schedules
 - Recovery is possible
- Nonrecoverable schedules should not be permitted by the DBMS
- No committed transaction ever needs to be rolled back
- ***Cascading rollback*** is a situation: where an uncommitted transaction has to be rolled back because it read an item from a transaction that failed.
 - Cascading rollback may occur in some recoverable schedules

Characterizing Schedules Based on Recoverability (cont'd.)

- Cascadeless schedule
 - Avoids cascading rollback
- Strict schedule
 - Transactions can neither read nor write an item X until the last transaction that wrote X has committed or aborted
 - Simpler recovery process
 - Restore the before image

Characterizing Schedules Based on Serializability

- Serializable schedules
 - Always considered to be correct when concurrent transactions are executing
 - Places simultaneous transactions in series
 - Transaction T_1 before T_2 , or vice versa

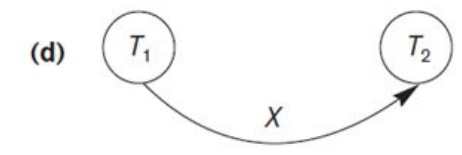
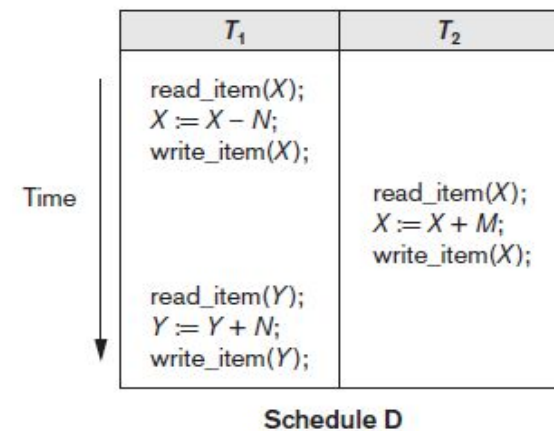
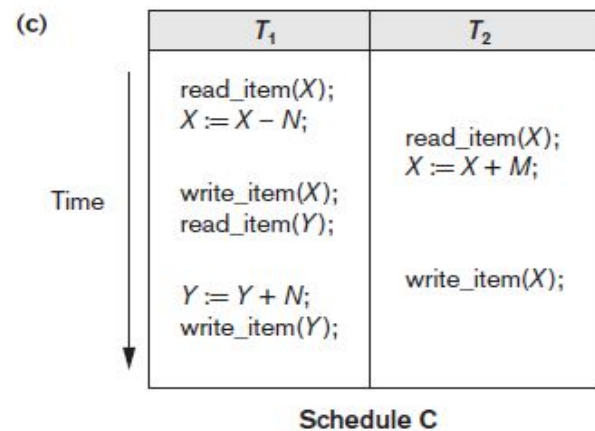
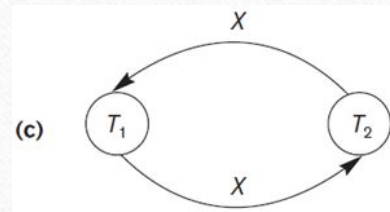
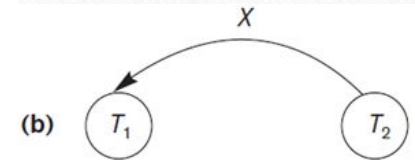
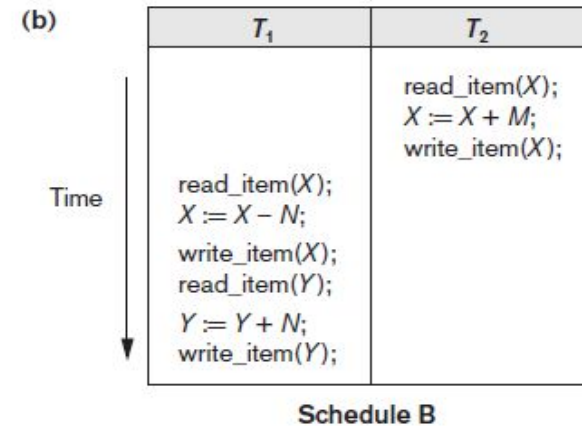
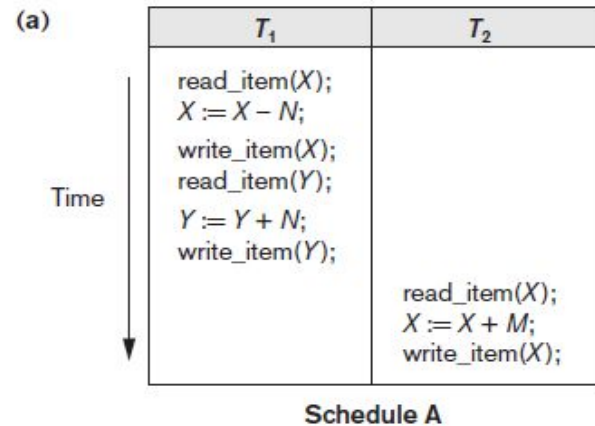
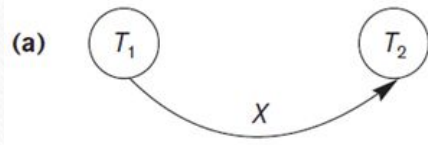


Figure 20.5 Examples of serial and nonserial schedules involving transactions T_1 and T_2 (a) Serial schedule A: T_1 followed by T_2 (b) Serial schedule B: T_2 followed by T_1 (c) Two nonserial schedules C and D with interleaving of operations

Characterizing Schedules Based on Serializability (cont'd.)

- Problem with serial schedules
 - Limit concurrency by prohibiting interleaving of operations
 - Unacceptable in practice
 - Solution: determine which schedules are equivalent to a serial schedule and allow those to occur
- Serializable schedule of n transactions
 - Equivalent to some serial schedule of same n transactions

Characterizing Schedules Based on Serializability (cont'd.)

- Conflict equivalence
 - Relative order of any two conflicting operations is the same in both schedules
- Serializable schedules (Called conflict serializable)
 - Schedule S is serializable if it is conflict equivalent to some serial schedule S'.

Characterizing Schedules Based on Serializability (cont'd.)

- Testing for serializability of a schedule
 1. For each transaction T_i participating in schedule S , create a node labeled T_i in the precedence graph.
 2. For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
 3. For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
 4. For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
 5. The schedule S is serializable if and only if the precedence graph has no cycles.

Algorithm 20.1 Testing conflict serializability of a schedule S

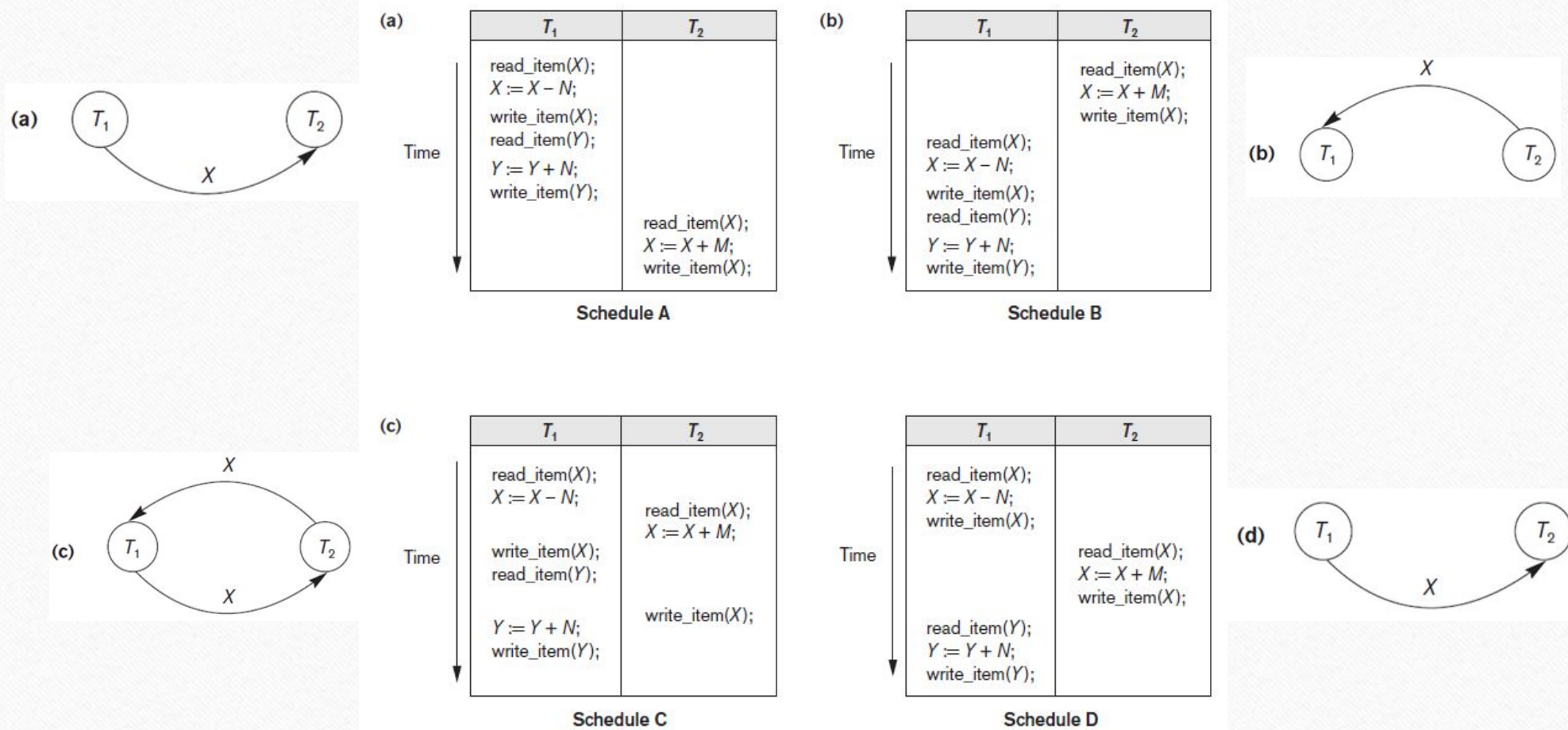


Figure 20.7 Constructing the precedence graphs for schedules A to D from Figure 20.5 to test for conflict serializability (a) Precedence graph for serial schedule A (b) Precedence graph for serial schedule B (c) Precedence graph for schedule C (not serializable) (d) Precedence graph for schedule D (serializable, equivalent to schedule A)

How Serializability is Used for Concurrency Control

- Being serializable is different from being serial
- Serializable schedule gives benefit of concurrent execution
 - Without giving up any correctness
- Difficult to test for serializability in practice
 - Factors such as system load, time of transaction submission, and process priority affect ordering of operations
- DBMS enforces protocols
 - Set of rules to ensure serializability

View Equivalence and View Serializability

- View equivalence of two schedules
 - As long as each read operation of a transaction reads the result of the same write operation in both schedules, the write operations of each transaction must produce the same results
 - Read operations said to see the same view in both schedules
- View serializable schedule
 - View equivalent to a serial schedule

(a)

Transaction T_1
read_item(X);
write_item(X);
read_item(Y);
write_item(Y);

Transaction T_2
read_item(Z);
read_item(Y);
write_item(Y);
read_item(X);
write_item(X);

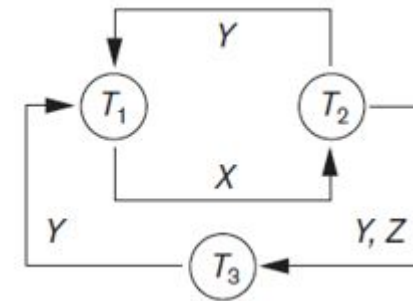
Transaction T_3
read_item(Y);
read_item(Z);
write_item(Y);
write_item(Z);

(b)

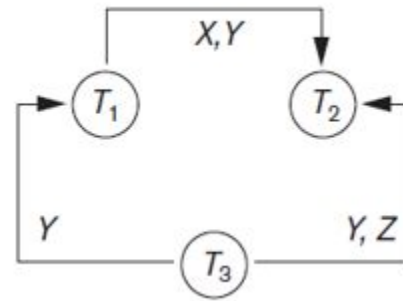
	Transaction T_1	Transaction T_2	Transaction T_3
Time ↓	read_item(X); write_item(X);	read_item(Z); read_item(Y); write_item(Y);	read_item(Y); read_item(Z);
	read_item(Y); write_item(Y);	read_item(X); write_item(X);	write_item(Y); write_item(Z);

Schedule E

(d)



(e)



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

(c)

Time
↓

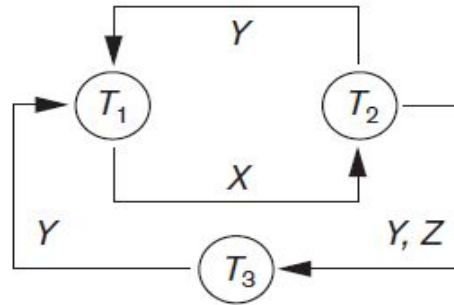
Transaction T_1	Transaction T_2	Transaction T_3
read_item(X); write_item(X);		read_item(Y); read_item(Z);
	read_item(Z);	write_item(Y); write_item(Z);
read_item(Y); write_item(Y);	read_item(Y); write_item(Y); read_item(X); write_item(X);	

Schedule F

Figure 20.8 (continued)

Another example of serializability testing. (d) Precedence graph for schedule E. (e) Precedence graph for schedule F. (f) Precedence graph with two equivalent serial schedules.

(d)



Equivalent serial schedules

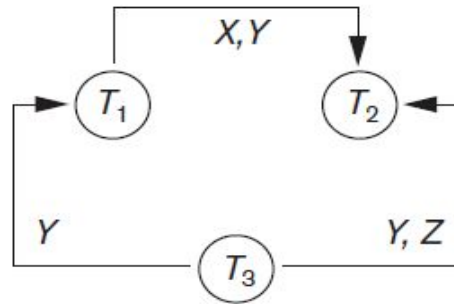
None

Reason

Cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$

Cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$

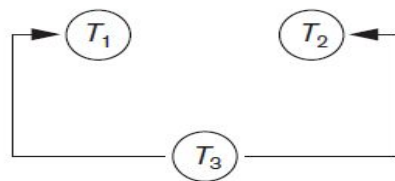
(e)



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

(f)



Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

$T_3 \rightarrow T_2 \rightarrow T_1$